ChatGPT

Diagnosing the Noise Issue in Class-Conditional Diffusion

Symptom: Generated Outputs are Unstructured Noise

The core problem is that at inference time, the diffusion model is producing images that look like **pure noise** (essentially garbage), with no distinguishable features. This occurs **consistently across all class indices** – i.e. every class-conditioned sample appears similarly noisy, and there's little to no variation between classes. In a correct class-conditional model, we'd expect each class to yield different-looking images (especially since your training visuals suggest the model *was* exposed to distinct class features). The fact that *all* outputs are similar noise indicates a fundamental issue in either the **sampling procedure** or the **model's learned conditional behavior**.

Before diving into solutions, it's important to pinpoint *why* this is happening:

Potential Causes of Failure

1. Sampling Procedure Mismatch or Bugs

Diffusion models require careful alignment between training and sampling. If the inference process (number of timesteps, beta schedule, etc.) doesn't exactly match the training configuration, the results can degrade into noise. In your case, you **did use the same sampling strategy and number of timesteps** (as confirmed by the config). For example, the scheduler is initialized with the same parameters (`num_train_timesteps=1000`, linear β schedule from 1e-4 to 0.02, `prediction_type="epsilon"`) and then set to 1000 inference steps, matching training [1]. This parity is good – any mismatch here could have been catastrophic (e.g. using far fewer steps than training without adjustment). So a *schedule mismatch* is likely **not** the culprit.

However, there was a known **sampling bug** in the code that could exactly produce the behavior you observed. Specifically, in the `visualize_denoising_process()` function (used during training and possibly in generation), passing `num_steps=0` was intended to skip intermediate frames and only get the final image. But due to a logic bug, it **returned only the initial noise sample and never appended the final denoised image** [2]. In other words, whenever the code tried to generate a sample without intermediate steps, it would inadvertently give you pure noise as the output (since the "output" was the starting noise) [3]. This bug affected the training visualizations (`epoch_XXXX_classes.png` looked like noise) **and** the generation script outputs [4] [5].

If you have not pulled in the fix for this bug, it could be the smoking gun. The fix (applied on 2025-10-21) ensures that the final denoised image is always appended and returned even when no intermediate steps are saved [6]. After patching, the training visuals should show meaningful images, and the generation script should produce proper outputs for each class [7]. In summary, double-check that

your code includes this correction – without it, the model *might* have learned fine, but you'd never see its real outputs (only the initial noise). This is a critical first step: a purely technical issue that masquerades as a model failure.

2. Classifier-Free Guidance Configuration

Your model is a **classifier-free guided diffusion**: it was trained by randomly dropping out the class conditioning on a fraction of training examples (you set `guidance_p_uncond = 0.1`, meaning 10% of the time no class label is provided) [8]. This teaches the model an unconditional generation ability alongside the conditional one. To leverage this at inference, one typically uses a **guidance scale > 1.0**, which blends the conditional and unconditional predictions to amplify class-specific features [9].

However, by default your config's `ccddpm.infer.guidance_scale` was `0.0` (if not overridden) [10]. A **guidance_scale of 0** means you are effectively telling the model to ignore the class label during generation – it uses the *unconditional* path only. In the generation code, this is exactly what happens: because `guidance_scale != 1.0`, the code branches to compute `eps = eps_uncond + scale * (eps_cond - eps_uncond)`. With scale=0, this simplifies to `eps = eps_uncond` [9], i.e. the model is denoising as if **no conditioning** is present for all samples. Naturally, if you generate unconditionally, you'll get similar outputs regardless of the class index. This perfectly matches the symptom: all classes coming out looking the same (and mostly noise).

Solution: use a non-zero guidance scale for generation. Even **guidance_scale = 1.0** would switch to *pure conditional* generation (using the class label without extra push). Values >1.0 provide stronger guidance. The project docs recommend around **2.0** as a good default [11], and indeed classifier-free guidance literature (Ho & Salimans, 2022) found that moderate guidance values can improve sample fidelity significantly [12]. Setting `guidance_scale` to 2–3 will force the model to more strongly adhere to the conditioning, often yielding much clearer, class-distinct images (at the cost of some diversity) [13]. In practice, you might want to experiment (e.g. 1.0 vs 2.0 vs 3.0) once the model is working, but **any** of these is better than 0! The key point: **a guidance scale of 0 was effectively collapsing your conditional generation into an unconditional one** – a likely reason all outputs were "garbage" and identical.

3. The Model Didn't Learn the Conditional Distribution (Training Issues)

If the sampling code and guidance configuration are correct, we have to consider that the model itself may not have learned to generate meaningful images. Diffusion models can fail to train for subtle reasons. In your training logs (before the recent fixes), there was evidence of a serious problem: the model's output was almost an **identity mapping of the noise**. The "input-output correlation" metric was extremely high (~0.99) [14], meaning instead of learning to remove noise, the network was basically returning the noisy input ("echoing" it). This would result in outputs that remain noisy at all timesteps. The likely cause was identified as **over-aggressive gradient clipping** and potentially other hyperparameter issues [15]. With `grad_clip_norm = 1.0`, the gradients were being clipped so much that the model couldn't effectively learn the denoising function, getting stuck in a local minimum (the near-identity function) [16].

You've already applied fixes for this: the clip norm was relaxed to 10.0 [17], and an important training technique called **Min-SNR loss weighting** was enabled [18]. Min-SNR weighting (Hang *et al.*, 2023) dynamically rescales the loss for each diffusion timestep, preventing the model from neglecting either the very early noise-heavy steps or the late nearly-clean steps [19]. This strategy balances the training across

timesteps and **improves convergence speed and stability** [20] [21]. By setting `use_min_snr: true` and `min_snr_gamma: 5.0` (a typical value), your training should now be more robust to the kind of instability you experienced. In essence, the model is encouraged to **actually learn denoising** rather than lazily outputting the input.

Despite these fixes, if the model was mid-training or near-converged *before* the fixes, it might not automatically unlearn bad behavior. You mentioned epoch 40 outputs – if those were still largely noise, it's possible the model had not yet recovered from the earlier phase or wasn't trained long enough with the new settings. In the documentation, a **"conditioning gap"** metric is discussed: this is the difference in model output when given the true class label versus when given a null (unconditional) label for the same input [22]. A healthy conditional model will show a growing gap (the model reacts to labels), whereas a gap near zero means the model is effectively ignoring the label [23]. Seeing "all classes identical" in outputs is a red flag that *conditioning didn't take*. If you logged the `gap_mean` during training (every 10 epochs per the docs), check its values [24]. If it stayed near 0, that confirms the model never really learned class differentiation. The fixes you applied (bug patch + Min-SNR + higher clip norm) set the stage for the model to learn conditioning, but they might need to be followed by sufficient training epochs to actually realize those gains.

In summary, **training issues to double-check**: Did the model get enough training *after* fixes? Are metrics like input-output correlation now back to a healthy range (<0.5) [25], and is the conditioning gap rising? If not, the model might require further training or tuning (more epochs, or adjustments like a larger class embedding size, etc., though your current embedding of 16 for 9 classes is likely fine).

Solutions and Recommendations

Based on the causes above, here's a comprehensive plan combining **code fixes**, **configuration changes**, and **methodological improvements** to get meaningful class-conditioned generations:

A. Fix Sampling and Leverage Guidance

- **Apply the Latest Code Fixes:** Make sure your code includes all the recent patches from the `medsyn` repo. In particular, the fix to always return the final image in `visualize_denoising_process()` is essential [6]. Without it, any samples generated during training or via the `ccddpm-generate` script might just be showing you the starting noise. The documentation explicitly notes that after this fix, `epoch_XXXX_classes.png` should display *distinct, denoised images* for each class (not uniform noise) [7]. Likewise, the generation script, when run on a good checkpoint, is expected to output realistic medical images rather than noise [26]. Double-check by running a quick generation of a few samples and inspecting the logs: the log should report final pixel value ranges around -1 to 1 (e.g. `Final image: range=[-0.8, 0.9]` is good) [27]. If you still see something like `range=[-2.5, 2.8]` (full noise range) [28], that indicates the model output is essentially unbounded noise – a sign that either the bug persists or the model is badly trained.

- **Use Classifier-Free Guidance at Inference:** As discussed, a `guidance_scale` of 0 was a mistake for conditional generation. Update your YAML config's `ccddpm.infer.guidance_scale` to a value like **2.0** (the repo's guide suggests 2.0–2.5) [11]. Then regenerate images. With this change, the

code will perform the two forward passes (conditional and unconditional) and mix them, instead of defaulting to unconditional-only [9]. You should immediately see a difference – even if images are still not perfect, samples from different classes should start to diverge in appearance when guidance is enabled. Ho & Salimans (2022) demonstrated that classifier-free guidance can substantially improve fidelity and class consistency of diffusion outputs [12], which is likely crucial for your case. Just beware that too high a guidance scale (e.g. >>4) can reduce diversity or re-introduce some artifact noise [13], but moderate values are very beneficial.

- **Match Training and Sampling Settings:** It sounds like you already do, but ensure no other discrepancies. For instance, if at training time you used an **EMA (Exponential Moving Average) of weights**, you want to use those EMA weights for generation. Your generation code smartly loads `state['ema']` if available [29] [30]. If your checkpoint's EMA was empty (due to the earlier bug where it wasn't saved [31]), then you'd be using non-EMA weights which might be a bit weaker. After retraining/fixing, you should have EMA weights; always prefer them for sampling – they typically yield smoother results because they average out noisy updates (this is evident in many diffusion works, where EMA models consistently improve FID/Inception scores). The log will tell you "Using EMA weights for generation (higher quality)" if it found them [29].

- **Verify the Noise Scheduler Usage:** You're using HuggingFace's `DDPMScheduler` for inference, which is good. It ensures the **same β schedule** and approach as training. The code sets `scheduler.set_timesteps(num_inference_steps)` with `num_inference_steps = 1000` (matching train) [32] [33], meaning it does an ancestral DDPM sampling with the full chain. That is the most faithful way to sample from a DDPM. For now, stick with this (1,000 steps, ancestral sampling) to troubleshoot quality. Later, once images look reasonable, you can consider faster samplers (like DDIM or DPM-Solver) to trade off speed vs. quality. The key is: **no discrepancy** between train and test schedule. If you ever change one, change the other to match (or use a known procedure for e.g. training with 1000 steps but sampling with fewer using "skip" techniques [34]).

B. Ensure the Model Learned Properly

- **Continue Training with Patches:** If your current model (epoch ~40 or even 90) was largely trained with the bugs, it may not have learned much. The project documentation explicitly calls out that prior to fixes, the model *had* effectively failed (echoing input, ignoring labels) [35] [22]. After applying fixes, it's recommended to **resume training** from your latest checkpoint [36] or even start fresh. The good news is that with the bug fixes:
- The model will actually be *able* to show its generative capability (since sampling now works). This means you can visually monitor progress. By epoch 10 or 20, you should start seeing *some* structure in `epoch_XXXX_classes.png` if things are on track [37]. All-nine-panels being pure noise after many epochs is a failure sign.
- The **EMA weights** are now properly tracked (that one-line change in the EMA initialization fixed the empty EMA issue) [38]. This will help training stability and final quality. Verify after training that your checkpoint's `state['ema']` has a reasonable size (the log snippet suggests ~331 parameters for this model) [31].
- The gradient clipping at 10.0 allows the model to make larger weight updates. Monitor the logged gradient norms: if you consistently see the grad norm hitting exactly 10.0 every step and being clipped, that might mean 10.0 is still restrictive, but chances are it will now hover below that once the model escapes the bad local minimum [39]. You could even disable grad clipping entirely as an

experiment if stability permits, but the current choice (10.0) was based on empirical evidence that 1.0 was too low [16].

- **Min-SNR loss weighting** is now active. This should help the model learn better features for both early and late timesteps. In diffusion, often the highest noise timesteps (t near T) and lowest noise timesteps (t near 0) are hard to balance – the model can otherwise over-focus on mid-range noise levels. Hang *et al.* (2023) introduce this weighting to clamp the SNR (signal-to-noise ratio) used in loss calculations, essentially preventing very large or very small SNR values from dominating [19]. The result is faster convergence and improved sample quality at given epochs [21]. Since you enabled it, you might notice your **loss curves** behave differently (possibly lower variance across epochs). The ultimate test is improved sample fidelity – which you'll gauge from the images.

- **Monitor Diagnostic Metrics:** Keep an eye on the **"conditioning gap"** and **input-output correlation** during training. The training code now logs these diagnostic metrics every epoch or so [40]. A quickly rising conditioning gap (from ~0 up to some higher value) means the model is learning to differentiate classes [24]. If it stays flat ~0, something is still wrong (e.g., labels not being used, or perhaps all labels in the data are the same due to a data issue). The input-output correlation should drop from ~1.0 down to a much lower number – ideally even negative (meaning the model is *inversely* correlating with input noise, as a proper denoiser would) [25]. In healthy diffusion models, this correlation starts high (the model initially doesn't denoise much) but should fall well below 0.5 as training progresses [41]. The **"prediction std"** (standard deviation of the model's predicted noise) is another useful metric – it should be on the order of 1.0 for normalized images [42]. If it's near 0, the model's outputs collapsed; if it's huge, the model might be blowing up. These diagnostics will alert you early if things are going awry, so you don't have to wait until many epochs of bad training.

- **Check Your Dataset and Preprocessing:** On the implementation side, verify that the class labels are correctly fed in. The `num_classes` in the config is 9 [43], and presumably classes are labeled 0–8. Ensure that the label IDs in your dataset actually match this (off-by-one errors can happen, e.g. if labels were 1–9 instead of 0–8). Given the context (PathMNIST has 9 tissue classes), it's likely fine. Also ensure that normalization is correct – the config says `normalize: True` which scales images to [-1,1] [44]. The generation code later clamps and un-normalizes with `(x+1)/2` to save images [45]. This is standard. Just be sure any image viewer you use isn't misinterpreting the format (saving with `torchvision.utils.save_image` after that normalization should yield a proper PNG). A mismatch in normalization could make even a perfectly good image appear as noise. The logs you have (showing ranges) are helpful here: if final images have range ~[-0.8,0.9] [27] and after normalization ~[0,1], they should look fine. If you saw ranges like [-10,10], that would indicate something seriously off (either in training or a bug in the scheduler usage).

C. Leverage Proven Diffusion Implementations (Optional)

Since you're open to switching libraries or approaches, it might be informative to **compare with a reference implementation**. OpenAI's *guided-diffusion* code (Dhariwal & Nichol, 2021) is a canonical implementation of class-conditional diffusion models that *"beat GANs"* on image synthesis [46]. It's not a drop-in replacement for your code, but here's how you could use it:

- **Train on a Small Sample with OpenAI's Pipeline:** Their repository (often referenced as *improved diffusion* or *guided diffusion*) has scripts to train diffusion models on CIFAR-10, ImageNet, etc., with

classifier guidance or classifier-free guidance. You could try training a smaller model on your dataset using that code. If it produces good outputs, then the issue was likely in the nuances of your implementation. If it also produces noise, then the problem might be data-related or something fundamental. Essentially, it's a way to sanity-check the task itself with a known-good codebase. Dhariwal & Nichol's implementation includes best practices like:

- Multi-head attention at multiple resolutions (your UNet has attention at the 32x32 level per `AttnDownBlock2D` in one stage [47], which is good – ImageNet models often put attention in higher resolutions too, but for 128×128 one attention layer might be okay).
- A learning rate schedule (they often use cosine decay or warmup + decay). Check if your training is using any LR schedule; if not, you might incorporate one to stabilize later training.
- They also used **variational lower-bound loss** components in training (to account for the diffusion prior term), but many implementations, including yours, stick to the simplified MSE loss on noise which is usually fine.
- OpenAI's code supports classifier guidance (i.e. training a separate classifier). That's not needed here since you have classifier-free guidance.

Even if you don't fully switch, **borrowing hyperparameters** from their success on similar tasks could help. For example, for ImageNet 128x128, they used a U-Net of a certain size, certain dropout, etc. Your model has `model_channels=128` and depth 4 with 2 res blocks each [48] [49] – that's a reasonable architecture for 128×128 images. You might compare with their config; if they used a larger network or more attention, it might inspire an architecture tweak if needed for quality.

- **Hugging Face Diffusers Integration:** You're already using some Diffusers components (UNet2DModel and DDPMScheduler), which is great for compatibility. Hugging Face's Diffusers library doesn't have a full training pipeline built-in for custom datasets (beyond example scripts), but it *does* offer many sampler algorithms and utilities for inference. Once your model is working, you could experiment with Diffusers' variety of schedulers (like DDIM, PNDM, or DPMSolver). These can often produce similar quality with fewer steps. For instance, DPMSolver++ can achieve excellent quality in ~20–50 steps for well-trained models. Initially, though, stick to the fully trained 1000-step process to validate the model itself.

- **Libraries for Advanced Features:** If you consider restructuring, another library is OpenAI's newer **guidance** code (which you have) and there's also the LAION's latent diffusion (if you ever move to latent space like Stable Diffusion did). Since you specifically mentioned guided-diffusion, it suggests you might try that first. Just be prepared that guided-diffusion isn't in PyTorch Lightning; you'd be writing some glue code to use its training loop or converting its model checkpoints to your format if needed. It's a trade-off: your current pipeline is neatly integrated with Lightning and your data format (with JSON indexes, etc.), whereas guided-diffusion will require adaptation. It *can* be done if the benefits outweigh the cost – e.g., if after all fixes your model still underperforms, then using the reference implementation to achieve a result might be worth it.

D. Final Checklist for Implementation

To wrap up, here's a step-by-step **action plan**:

1. **Update and Re-run Generation (Short-Term Test):** Apply the bug fix and set `guidance_scale=2.0` in your YAML. Use your latest checkpoint and run `ccddpm-generate`. This quick test will tell you if things are already resolved:
2. If you see **structured images** (even if blurry or imperfect) that differ per class, you've likely solved the immediate issue (the model was fine; it was the code/guidance misuse that was wrong).
3. If you still see pure noise everywhere, then the model probably never learned. In that case, move to step 2.
4. **Resume Training with Fixes:** Continue training from the checkpoint (or start anew if the checkpoint is suspect) using the fixed code. Monitor the training logs closely:
5. Watch for improvements in the diagnostics (lower correlation, higher conditioning gap).
6. Every few epochs, inspect the sample images logged. They should start as fuzzy blobs but gradually acquire some class-specific features (for PathMNIST, maybe color hues or texture unique to each tissue class).
7. Utilize early stopping or manual inspection. The docs mention an early stopping patience of 15 epochs [50] – ensure this didn't cut off training prematurely if the model hadn't improved due to the earlier issues. It might be wise to override patience or monitor manually until you are confident in progress.
8. **Validate Generation Quality:** After training (or periodically at checkpoints), generate images with a moderate guidance scale (1.0 and 2.0, for example) and evaluate them:
9. Do they resemble the real data distribution (e.g., **"tissue structures visible, colors make sense"** as noted in your guide) [27] ?
10. Are different classes **visibly distinct**? If yes, conditioning succeeded. If still not, there may be a deeper issue (perhaps the data itself is hard, or the model capacity is too limited).
11. Check the value ranges in logs for final images. They should be nicely bounded in [-1,1] (with `clip_sample=True` you have that) [51] . Extremely out-of-range values would indicate divergence.
12. **Fine-Tune and Experiment:** Once things are basically working, you can tweak:
13. Increase the model size or embedding dims if you think more capacity is needed.
14. Try alternative beta schedules (diffusers offers `beta_schedule="squaredcos_cap_v2"` which is a cosine schedule that Karras *et al.* (2022) found helpful for certain models [52]).
15. Experiment with **v-prediction** (predicting the velocity instead of noise). Some recent models (Stable Diffusion 2) use *v* as it can stabilize training for high resolution. Diffusers' scheduler and UNet support `prediction_type="v_prediction"` if configured. This would require adjusting loss calculation accordingly. It's an advanced tweak, but mentioning it for completeness – if your model still had trouble, this is an avenue to explore (as hinted by research on improved objectives [53]).

16. Adjust the **guidance scale** at inference per class if needed. Sometimes one class might need a slightly different scale to look right, especially if the dataset is imbalanced. Higher guidance can compensate for underrepresented classes by forcing the model to stick to the conditioning signal [13].

17. **Refer to Literature:** Continue to draw inspiration from research:

18. *Classifier-Free Guidance* paper by Ho & Salimans (2022) is a short but insightful read on how guidance balances diversity vs. fidelity [12]. It reassures that your approach (joint conditional/unconditional training) is sound and in fact the go-to method for text and class conditional diffusion models today.
19. *Efficient Diffusion Training via Min-SNR* by Hang *et al.* (2023) [20] provides the theoretical reasoning for the weighting trick you're using. Understanding it can help you explain to others why your model converges faster or why you chose certain γ value.
20. Dhariwal & Nichol (NeurIPS 2021) "Diffusion Models Beat GANs" details the results of class-conditional diffusion on ImageNet and other benchmarks [46]. They also open-sourced their code which you're considering; their paper's ablations might guide your hyperparameter choices (e.g., they discuss how guidance or classifier guidance impacts FID).
21. The community has also produced *troubleshooting guides* for diffusion models (since these models are finicky). Even your own repository's **BUG_FIX_TESTING_GUIDE** and similar docs are gold – they capture domain-specific gotchas (like "all classes identical" as a failure sign of conditioning) [22] that not every generic paper will mention.

By systematically applying these fixes and improvements, you should move from "garbage/noise at inference" to **meaningful, class-conditioned images**. In summary, the likely issue was a combination of a code bug and configuration (guidance scale) rather than a fundamental flaw in the diffusion model. Once those are addressed and the model is properly trained to convergence, diffusion models are capable of generating sharp and diverse images even in challenging domains. Good luck, and happy diffusing!

Sources:

- Project documentation on diffusion model training fixes and diagnostics [14] [15] [42].
- Project documentation on the generation bug and its resolution [54] [6].
- Quick start guide highlighting expected outcomes vs. failure cases (post-fix) [27] [22].
- Classifier-free guidance method by Ho & Salimans (2022) – combining conditional and unconditional diffusion outputs to improve fidelity [12].
- Min-SNR loss weighting strategy by Hang *et al.* (2023) for faster, more stable diffusion training [19] [20].
- Dhariwal & Nichol (2021), *Diffusion Models Beat GANs*, for reference class-conditional diffusion implementation and results [46].

---

[1] [9] [29] [30] [32] [33] [45] [51] generate_ccDDPM.py
https://github.com/MarioPasc/medsyn/blob/80d152a664c1abd66ca469d6fec211de47c18588/medsyn/cli/generate_ccDDPM.py

[2] [3] [4] [5] [6] [7] [54] CCDDPM_GENERATION_BUG_FIX.md
https://github.com/MarioPasc/medsyn/blob/80d152a664c1abd66ca469d6fec211de47c18588/docs/CCDDPM_GENERATION_BUG_FIX.md

8 50 train_ccDDPM.py
https://github.com/MarioPasc/medsyn/blob/80d152a664c1abd66ca469d6fec211de47c18588/medsyn/cli/train_ccDDPM.py

10 43 44 52 config.py
https://github.com/MarioPasc/medsyn/blob/80d152a664c1abd66ca469d6fec211de47c18588/medsyn/models/ccDDPM/config.py

11 13 22 23 24 26 27 28 36 37 QUICK_START_AFTER_PATCHES.md
https://github.com/MarioPasc/medsyn/blob/80d152a664c1abd66ca469d6fec211de47c18588/docs/QUICK_START_AFTER_PATCHES.md

12 [2207.12598] Classifier-Free Diffusion Guidance
https://arxiv.org/abs/2207.12598

14 15 16 17 18 25 31 35 38 39 40 41 42 TRAINING_FIXES_AND_DIAGNOSTICS.md
https://github.com/MarioPasc/medsyn/blob/80d152a664c1abd66ca469d6fec211de47c18588/docs/TRAINING_FIXES_AND_DIAGNOSTICS.md

19 CCDDPM_CODE_PATCHES_APPLIED.md
https://github.com/MarioPasc/medsyn/blob/80d152a664c1abd66ca469d6fec211de47c18588/docs/CCDDPM_CODE_PATCHES_APPLIED.md

20 [PDF] Efficient Diffusion Training via Min-SNR Weighting Strategy
https://openaccess.thecvf.com/content/ICCV2023/papers/Hang_Efficient_Diffusion_Training_via_Min-SNR_Weighting_Strategy_ICCV_2023_paper.pdf

21 Efficient Diffusion Training via Min-SNR Weighting Strategy
https://www.researchgate.net/publication/377431217_Efficient_Diffusion_Training_via_Min-SNR_Weighting_Strategy

34 46 [PDF] Diffusion Models Beat GANs on Image Synthesis - NIPS papers
https://proceedings.nips.cc/paper/2021/file/49ad23d1ec9fa4bd8d77d02681df5cfa-Paper.pdf

47 48 49 model.py
https://github.com/MarioPasc/medsyn/blob/80d152a664c1abd66ca469d6fec211de47c18588/medsyn/models/ccDDPM/model.py

53 [PDF] IMPROVED NOISE SCHEDULE FOR DIFFUSION TRAINING
https://openreview.net/pdf?id=j3U6CJLhqw